Livraria API — perguntas prováveis (AV2)

JWT e segurança

1. O que é JWT?
É um token assinado, no formato `header.payload.assinatura`, cada parte em Base64. O header diz o algoritmo (HS256), o payload carrega as informações (as *claims*: aqui o e-mail no `sub`, a data de criação no `iat` e a de expiração no `exp`) e a assinatura garante que ninguém alterou o conteúdo.

2. Por que JWT e não sessão?
Com sessão, o servidor guarda na memória quem está logado. Isso não escala: com vários servidores, todos precisam compartilhar as sessões. Com JWT o servidor não guarda nada (stateless): cada requisição traz o token e se prova sozinha, que é o princípio do REST.

3. Diferença entre autenticação e autorização?
Autenticação é saber **quem** você é (login, token válido). Autorização é saber **o que** você pode fazer (ex: só ADMIN apaga). Neste projeto fiz autenticação: qualquer usuário logado acessa tudo. Autorização por papel seria o próximo passo.

4. Qualquer um consegue ler o token. Por que não dá pra trocar o e-mail e virar outro usuário?
Porque a assinatura é calculada com HMAC-SHA256 sobre o header e o payload usando a secret, que só o servidor conhece. Se alguém muda o payload, o servidor recalcula a assinatura, ela não bate com a recebida e a requisição dá 401. Sem a secret, não tem como gerar uma assinatura nova válida.

5. Por que não colocar a senha no token?
Base64 não é criptografia, é só codificação: qualquer pessoa decodifica e lê o payload. Por isso nada sensível vai no token, só o identificador do usuário e as datas.

6. Onde fica a secret e por quê?
No `application.yaml` ela é lida de uma variável de ambiente (`${JWT_SECRET:...}`), com um valor padrão só pra desenvolvimento. Em produção a secret real é passada pelo ambiente e nunca vai pro Git: se vazar, qualquer um consegue forjar tokens válidos.

7. O que acontece quando o token expira?
O `exp` fica 1 hora depois do login (`jwt.expiration: 3600000` ms). Depois disso, a validação no `JwtFilter` lança exceção, o usuário não é autenticado e a requisição recebe 401. Precisa fazer login de novo. Expiração curta limita o estrago se o token vazar.

8. O que o `JwtFilter` faz, passo a passo?
Ele estende `OncePerRequestFilter`, então roda uma vez por requisição, antes do controller. Pega o cabeçalho `Authorization`, tira o prefixo `Bearer`, valida assinatura e expiração pelo `JwtService` e, se estiver tudo certo, coloca o usuário no `SecurityContextHolder`. Se não tiver token ou ele for inválido, só segue sem autenticar.

9. Quem devolve o 401, o filtro ou o controller?
Nenhum dos dois. O filtro só autentica ou não. Quem barra é o Spring Security, pela regra `anyRequest().authenticated()` no `SecurityConfig`: se a requisição chega sem usuário autenticado, ele responde 401 antes de chegar no controller.

10. Por que desabilitar o CSRF?
CSRF é um ataque que se aproveita do cookie de sessão que o navegador manda sozinho. Esta API não usa cookie nem sessão: o token vai no cabeçalho, e o navegador não manda isso automaticamente. Então a proteção não se aplica.

11. O que significa STATELESS no `SecurityConfig`?
Que o Spring não cria nem usa sessão HTTP. Cada requisição é avaliada só pelo token que traz, sem depender de nada guardado no servidor.

12. Como a senha é guardada?
Com BCrypt, pelo `PasswordEncoder`. Ele gera um hash com *salt* aleatório, então duas pessoas com a mesma senha têm hashes diferentes, e não dá pra voltar do hash pra senha. No login, o `matches` aplica o mesmo processo à senha digitada e compara.

13. Por que o login responde a mesma mensagem pra e-mail errado e senha errada?
Pra não entregar a quem está tentando invadir quais e-mails existem no sistema. "E-mail ou senha inválidos" não diz qual dos dois errou.

14. Que rotas são públicas?
`/api/auth/register`, `/api/auth/login` e as do Swagger (`/swagger-ui/**` e `/v3/api-docs/**`). Todo o resto exige token.

15. Qual a diferença entre 401 e 403?
401 (Unauthorized) é "não sei quem você é": sem token ou token inválido. 403 (Forbidden) é "sei quem você é, mas você não pode fazer isso": autenticado, sem permissão. Aqui só existe 401, porque não há papéis; com USER e ADMIN, um USER tentando apagar receberia 403.

16. O que é o `SecurityContextHolder`?
É onde o Spring Security guarda quem está autenticado na requisição atual. O `JwtFilter` coloca o usuário lá, e as regras do `SecurityConfig` consultam dali. Ele é por requisição (por thread), então um usuário não "vaza" para a requisição de outro.

17. Por que o filtro estende `OncePerRequestFilter`?
Porque uma requisição pode passar pela cadeia de filtros mais de uma vez (num redirecionamento interno, por exemplo). O `OncePerRequestFilter` garante que a validação do token roda uma vez só por requisição.

18. O que é HMAC-SHA256 (HS256)?
Um algoritmo de assinatura com chave secreta: aplica o hash SHA-256 ao conteúdo misturado com a secret. Só quem tem a secret gera ou confere a assinatura. É simétrico: a mesma chave assina e valida.

19. Por que a secret precisa ser longa?
Para HS256 a chave precisa ter pelo menos 256 bits (32 bytes); a própria biblioteca jjwt recusa chaves menores. Chave curta ou previsível pode ser descoberta por tentativa e erro, e aí qualquer um forja tokens.

20. Dá pra "deslogar" com JWT?
Não de verdade: o token vale até expirar, porque o servidor não guarda estado. O logout no front é só apagar o token. Para invalidar antes, seria preciso uma lista de tokens bloqueados (o que volta a guardar estado) ou expiração curta com refresh token.

21. O que é refresh token?
Um segundo token, de vida longa, usado só para pedir um novo token de acesso quando o atual expira. Assim o token de acesso pode durar poucos minutos sem o usuário ter que fazer login toda hora. Não foi implementado aqui.

22. Onde o front guarda o token?
Normalmente na memória ou no `localStorage` do navegador e manda no cabeçalho `Authorization` a cada requisição. Por isso é importante usar HTTPS: quem interceptar o token se passa pelo usuário até ele expirar.

23. Por que HTTPS importa aqui?
O token e a senha do login viajam na requisição. Sem HTTPS, qualquer um na mesma rede consegue ler e reutilizar. Em produção a API tem que estar atrás de HTTPS.

24. O que é CORS? Precisou configurar?
É a regra do navegador que bloqueia um site de chamar uma API de outro domínio, a menos que a API permita. Como aqui os testes são pelo Swagger (mesmo endereço) e pelo terminal, não foi preciso. Com um front em outro endereço (ex: React em `localhost:3000`), teria que liberar no `SecurityConfig`.

25. O que é BCrypt e por que não SHA-256 pra senha?
SHA-256 é rápido, e isso é ruim para senha: dá para testar bilhões de tentativas por segundo. O BCrypt é lento de propósito (tem um custo configurável) e usa *salt* aleatório, então ataques de força bruta e tabelas prontas de hashes ficam inviáveis.

26. O que é *salt*?
Um valor aleatório misturado à senha antes de gerar o hash e guardado junto com ele. Faz duas senhas iguais terem hashes diferentes e impede o uso de tabelas pré-calculadas.

27. O que está dentro do token que a API gera?
Header `{"alg":"HS256"}`, payload `{"sub":"email do usuário","iat":...,"exp":...}` e a assinatura. Dá para ver colando o token no site jwt.io: o conteúdo aparece, mas sem a secret não dá para gerar uma assinatura válida.

28. O usuário que faz login pode acessar tudo?
Sim. Qualquer usuário autenticado acessa todas as rotas, porque não há papéis (roles). Para restringir, seria adicionar um campo `role` no usuário, colocar no token e usar regras como `hasRole("ADMIN")` no `SecurityConfig`.

Flyway e banco

29. O que é uma migration?
Um arquivo versionado que descreve uma mudança no banco: criar tabela, adicionar coluna, criar índice. Aplicadas em ordem, elas recriam o banco igual em qualquer máquina: dev, teste, produção ou a de um colega novo.

30. Como o Flyway sabe o que já rodou?
Ele cria a tabela `flyway_schema_history` e anota cada migration aplicada (versão, descrição, checksum e se deu certo). Ao subir, compara a pasta com essa tabela e roda só as novas.

31. Por que `validate` e não `update`?
Com `update` o Hibernate altera o banco sozinho, sem histórico e com risco de perder dados. Com `validate` ele só confere se as entidades batem com as tabelas e, se não baterem, a aplicação não sobe. Quem muda o banco é só o Flyway; se os dois mexessem, brigariam.

32. Se precisar adicionar uma coluna, edita a V1?
Não. Migration aplicada nunca se edita: o Flyway guarda o checksum e acusa erro se o arquivo mudar, e os outros ambientes já rodaram a versão antiga. Cria uma nova, a V6, com `ALTER TABLE`.

33. Por que o nome do arquivo é `V1__create_table_autor.sql`?
É a convenção do Flyway: `V` de versionada, o número da versão, **dois** underscores e a descrição. Com um underscore só ele ignora o arquivo.

34. Por que o UUID não é gerado no SQL?
Quem gera é o Java, pelo `@GeneratedValue(strategy = GenerationType.UUID)`: o Hibernate cria o id antes do `INSERT`. No SQL a coluna só é declarada como `UUID PRIMARY KEY`.

35. O que é o checksum do Flyway?
Um número calculado a partir do conteúdo de cada migration e guardado na `flyway_schema_history`. Se alguém editar um arquivo já aplicado, o checksum muda e o Flyway acusa erro ao subir, em vez de deixar o banco diferente do código sem ninguém perceber.

36. E se uma migration der erro no meio?
O Flyway para, marca a falha e a aplicação não sobe. Corrige-se o SQL (se ainda não foi aplicada com sucesso em nenhum ambiente) ou cria-se uma nova migration que conserta. Não se apaga a `flyway_schema_history` na mão.

37. Dá pra desfazer (rollback) uma migration?
Não automaticamente na versão gratuita. O jeito é criar uma nova migration que desfaz a mudança (ex: V7 que remove a coluna que a V6 criou). O histórico continua andando só para frente.

38. Por que escrever SQL na mão em vez de deixar o Hibernate gerar?
Controle e transparência: o que está no arquivo é exatamente o que roda no banco. E fica no Git, com histórico de quem mudou o quê.

39. Por que H2? Funcionaria com PostgreSQL?
O H2 em memória é prático para desenvolver e apresentar: não precisa instalar nada. Para PostgreSQL, bastaria trocar o driver e o `datasource` no `application.yaml` e adicionar a dependência `flyway-database-postgresql`; as migrations são SQL padrão e rodariam igual.

40. Por que UUID e não um número sequencial (`Long`)?
O UUID é gerado sem consultar o banco e não é previsível: ninguém descobre quantos registros existem nem chuta o próximo id (`/livros/1`, `/livros/2`...). A desvantagem é ocupar mais espaço e ser menos legível.

41. O que é o console do H2?
Uma página web (`/h2-console`) para ver as tabelas e rodar SQL no banco em memória. Com o Spring Security ligado, ele também passa a exigir autenticação.

Modelo e relacionamentos

42. O que é chave estrangeira e o que ela garante?
É uma coluna que guarda o id de uma linha de outra tabela (`livro.autor_id` → `autor.id`). O banco garante a integridade: não aceita um livro com autor inexistente e não deixa apagar um autor que ainda tem livro.

43. O que fazem `@ManyToOne` e `@JoinColumn`?
`@ManyToOne` diz que muitos livros apontam para um mesmo autor. `@JoinColumn(name = "autor_id")` diz qual coluna da tabela livro guarda essa ligação. No Java eu trabalho com o objeto `Autor`; o Hibernate grava e lê só o id.

44. Por que o JSON do livro recebe o ID do autor, e não o autor inteiro?
O autor já está cadastrado; o cliente só diz qual é. O `LivroService` busca pelo ID usando o `AutorService`, que lança exceção se não existir, e a resposta já sai 404.

45. Por que o service do livro usa o `AutorService` e não o `AutorRepository`?
Pra reaproveitar a regra que já existe: o `buscarAutor` já trata o "não encontrado". Assim a mensagem e o 404 ficam num lugar só.

46. Por que excluir um autor dá 409?
A chave estrangeira impede apagar um autor que tem livro apontando pra ele, e o banco lança erro de integridade. Tratei a `DataIntegrityViolationException` no handler global pra devolver 409 Conflict com uma mensagem clara, em vez de 500.

47. O livro pode ficar sem autor, categoria ou editora?
Não. As três colunas são `NOT NULL` na migration e os campos do DTO têm `@NotNull`. Se faltar, a validação responde 400 antes mesmo de chegar no banco.

48. O que é JPA e o que é Hibernate?
JPA é a especificação (as anotações `@Entity`, `@Id`, `@ManyToOne`... e as regras de mapeamento). Hibernate é a implementação que faz o trabalho de verdade: gera o SQL, converte linhas em objetos e vice-versa.

49. O que é ORM?
Object-Relational Mapping: mapear objetos Java para tabelas relacionais. Você trabalha com `livro.getAutor().getNome()` e o Hibernate cuida dos `SELECT`, `INSERT` e `JOIN`.

50. O `@ManyToOne` carrega o autor junto com o livro?
Sim. Por padrão o `@ManyToOne` é EAGER: ao buscar um livro, o autor vem junto. Por isso o mapper consegue fazer `livro.getAutor().getNome()` direto.

51. Como seria a relação do lado do autor?
Com `@OneToMany(mappedBy = "autor")` na classe `Autor`, uma lista dos livros dele. Não foi necessário: a API não precisa listar os livros de um autor, e a relação de um lado só é mais simples.

52. E se um livro tivesse vários autores?
Seria muitos-para-muitos (`@ManyToMany`), com uma tabela intermediária `livro_autor` (livro_id, autor_id). Na nossa modelagem cada livro tem um autor só.

53. Por que a resposta do livro mostra o nome e não o id do autor?
Para ser mais útil para quem consome: quem lista os livros quer ver "Machado de Assis", não um UUID. Se precisasse dos dois, o DTO de resposta poderia ter `autorId` e `autorNome`.

Arquitetura e erros

54. Como o projeto está organizado?
Em camadas: o controller recebe a requisição HTTP, o service tem a regra de negócio, o repository fala com o banco, o model é a entidade, os DTOs são o formato de entrada e saída e o mapper converte entre DTO e entidade. Cada camada tem uma responsabilidade.

55. Por que usar DTO e não devolver a entidade direto?
Pra controlar o que entra e o que sai. A entrada tem validação (`@NotBlank`, `@NotNull`) e não deixa o cliente mandar o id, por exemplo. A saída mostra o nome do autor em vez do objeto inteiro e nunca expõe dados internos, como a senha do usuário.

56. Quem gera o 404?
O service lança uma exceção (ex: `AutorNaoEncontradoException`) e o `GlobalExceptionHandler`, com `@RestControllerAdvice`, transforma em 404 com um JSON de erro padrão. O service não sabe nada de HTTP.

57. Quais status a API usa e quando?
200 (ok), 201 (criado, com cabeçalho `Location`), 204 (excluído, sem corpo), 400 (dados inválidos), 401 (sem token ou token inválido), 404 (não encontrado), 409 (conflito: e-mail repetido ou registro vinculado) e 500 só pra erro inesperado.

58. Por que o DELETE de algo que não existe dá 404?
Antes dava 204 porque o `deleteById` não avisa quando o id não existe. Agora o service busca primeiro: se não achar, lança a exceção e vira 404. Assim o cliente sabe que não apagou nada.

59. O que é injeção de dependência?
Em vez de a classe criar o que precisa (`new LivroRepository()`), ela **pede no construtor** e o Spring entrega. Facilita trocar implementações e testar (dá para passar uma versão falsa num teste). É o que acontece em todos os construtores dos services e controllers.

60. Por que injeção pelo construtor e não `@Autowired` no campo?
Pelo construtor os campos podem ser `final` (não mudam depois), as dependências ficam explícitas e a classe pode ser criada num teste sem o Spring. É a forma recomendada.

61. Qual a diferença entre `@Component`, `@Service`, `@Repository` e `@RestController`?
Todas registram a classe para o Spring criar e injetar. Os nomes indicam a camada: `@Service` para regra de negócio, `@Repository` para acesso a dados, `@RestController` para rotas HTTP, `@Component` para o resto (ex: o mapper e o filtro).

62. O que é um `@Bean`?
Um método numa classe `@Configuration` que devolve um objeto para o Spring gerenciar. Foi usado no `SecurityConfig` (`SecurityFilterChain` e `PasswordEncoder`) e no `OpenApiConfig` (`OpenAPI`), porque são objetos de bibliotecas que a gente monta, não classes nossas.

63. Por que `record` nos DTOs?
O `record` (desde o Java 16) cria uma classe imutável só com os campos, já com construtor, getters (`dto.titulo()`), `equals` e `toString`. Perfeito para DTO, que só carrega dados.

64. Por que o mapper existe, em vez de converter no controller?
Para separar responsabilidades: o controller cuida de HTTP, o mapper da conversão, o service da regra. Fica mais fácil achar e mudar cada coisa, e o controller fica curto.

65. O que é o `Optional`?
Um tipo que representa "pode ter valor ou não". O `findById` devolve `Optional` para obrigar quem chama a tratar o caso vazio, e o `orElseThrow` transforma "vazio" em exceção. Evita `NullPointerException`.

66. Por que a validação fica no DTO e não na entidade?
Porque é a regra da **entrada**: o que o cliente pode mandar. Ela roda no controller (`@Valid`) e devolve 400 antes de chegar na regra de negócio. O banco ainda tem o `NOT NULL` como última proteção.

REST e HTTP

67. O que é uma API REST?
Uma API que organiza tudo em recursos (`/livros`, `/autores`), usa os verbos HTTP para as ações (GET, POST, PUT, DELETE), responde com os status HTTP e é stateless: cada requisição traz tudo o que precisa.

68. Qual a diferença entre PUT e PATCH?
PUT substitui o recurso inteiro (manda todos os campos), PATCH altera só alguns. Aqui o `PUT /livros/{id}` exige todos os campos, porque o DTO é validado completo.

69. O que é idempotência?
Fazer a mesma requisição várias vezes tem o mesmo efeito que fazer uma vez. GET, PUT e DELETE são idempotentes; POST não (cada POST cria um livro novo).

70. Por que o POST devolve 201 e o cabeçalho `Location`?
201 Created é o status certo para "criei um recurso", e o `Location` diz onde ele está (`/api/v1/livros/{id}`). É o padrão REST para criação.

71. Por que o DELETE devolve 204?
204 No Content: deu certo e não há nada para devolver no corpo.

72. Por que `/api/v1/` na URL?
O `v1` é a versão da API. Se um dia o formato mudar de um jeito que quebre quem já usa, cria-se `/api/v2/` e a v1 continua funcionando para os clientes antigos.

Swagger

73. Qual a diferença entre OpenAPI e Swagger?
OpenAPI é a especificação: um formato padrão (JSON/YAML) que descreve a API. Swagger são as ferramentas em volta dela, como a página interativa. O springdoc gera a especificação a partir dos controllers e mostra no Swagger UI.

74. O que fazem `@Tag`, `@Operation`, `@ApiResponses` e `@Schema`?
`@Tag` agrupa as rotas de um controller. `@Operation` descreve o que cada rota faz. `@ApiResponses` lista os status possíveis e o significado de cada um. `@Schema` descreve os campos do DTO e dá um exemplo, que já vem preenchido no "Try it out".

75. Como o Swagger usa o token?
No `OpenApiConfig` registrei um esquema de segurança bearer JWT, que cria o botão Authorize. Colando o token ali, o Swagger manda `Authorization: Bearer ...` em todas as requisições.

76. A documentação atualiza sozinha?
Sim. O springdoc lê os controllers e DTOs toda vez que a aplicação sobe. Mudou uma rota ou um campo, a página muda junto. Documento separado (Word, wiki) desatualiza.

77. O que é o `/v3/api-docs`?
A especificação OpenAPI em JSON, gerada pelo springdoc. A página do Swagger lê esse JSON para se montar, e outras ferramentas (ex: geradores de cliente, Postman) também conseguem importar.

78. Por que as rotas do Swagger estão liberadas no `SecurityConfig`?
Senão a própria página da documentação pediria token, e não daria para abrir, nem fazer o login por ela.

Docker

79. Qual a diferença entre imagem e container?
A imagem é o molde, montada pelo Dockerfile. O container é a imagem rodando. É como classe e objeto: de uma imagem dá pra subir vários containers.

80. Por que o Dockerfile tem duas etapas?
A primeira usa o JDK completo e o Maven pra compilar o `.jar`. A segunda usa só o JRE, que é menor, e copia apenas o `.jar`. A imagem final não carrega o compilador nem o código-fonte.

81. Por que copiar o `pom.xml` antes do código?
Cada linha do Dockerfile vira uma camada com cache. Baixando as dependências antes de copiar o `src`, uma mudança no código não obriga a baixar tudo de novo: o Docker reaproveita a camada das dependências enquanto o `pom.xml` não mudar.

82. Por que rodar com um usuário que não é root?
Se alguém explorar uma falha na aplicação, o processo não tem privilégio de administrador dentro do container. Limita o estrago.

83. Pra que serve o `compose.yaml`?
Guarda a configuração de como subir o container (de onde montar a imagem e qual porta expor), pra rodar tudo com um comando só: `docker compose up --build`. Em projetos maiores ele sobe vários containers juntos, como a API e o banco.

84. O que faz o `-p 8080:8080` (ou o `ports` do compose)?
Liga a porta do computador à porta do container: `porta_do_host:porta_do_container`. Sem isso a aplicação roda, mas fica inacessível de fora do container.

85. Qual a diferença entre container e máquina virtual?
A VM tem um sistema operacional inteiro com kernel próprio, então é pesada e demora para subir. O container compartilha o kernel do sistema hospedeiro e só isola os processos e arquivos: é leve e sobe em segundos.

86. O que é uma camada (layer)?
Cada instrução do Dockerfile gera uma camada da imagem. O Docker guarda as camadas em cache e só refaz a partir da primeira que mudou. Por isso a ordem das instruções importa.

87. Por que alpine?
É uma distribuição Linux mínima. A imagem com JRE alpine é bem menor do que a completa: download e deploy mais rápidos e menos coisa instalada para ter falha de segurança.

88. O que é o `docker.socket`?
Uma forma de o sistema iniciar o Docker sob demanda: o serviço só liga quando algum comando `docker` é usado, em vez de ficar rodando desde o boot.

89. Os dados ficam salvos no container?
Não. Aqui o banco é em memória, então some quando o container para. Mesmo com um banco em arquivo, o certo seria um *volume* (uma pasta do computador montada no container) para os dados sobreviverem quando o container é recriado.

90. Como subiria um PostgreSQL junto?
Adicionando um segundo serviço no `compose.yaml` (imagem `postgres`, com volume para os dados) e apontando o `datasource` da API para ele. O compose sobe os dois na mesma rede, e a API acessa o banco pelo nome do serviço.

Ferramentas

91. O que é o Maven e o `mvnw`?
Maven gerencia as dependências (baixa o que está no `pom.xml`) e o build (compila, testa, empacota o `.jar`). O `mvnw` (Maven Wrapper) baixa a versão certa do Maven sozinho, então o projeto roda em qualquer máquina sem instalar o Maven.

92. O que é o `.jar` gerado?
O pacote da aplicação com tudo dentro, inclusive o Tomcat embutido. Roda com `java -jar app.jar`, sem precisar instalar servidor.

93. Por que Java 25?
É a versão LTS (suporte longo) atual, e o projeto foi criado com ela no Spring Boot 4. Precisa de Java 25 instalado para compilar fora do Docker; no Docker, a imagem já traz.

Dificuldades e próximos passos

94. Teve alguma dificuldade?
O slide do JWT usa a dependência `io.jsonwebtoken:jjwt:0.11.5`, que não existe no repositório Maven. Nessa versão a biblioteca é dividida em três: `jjwt-api`, `jjwt-impl` e `jjwt-jackson`.

95. O que você melhoraria?
Autorização por papel (USER e ADMIN), trocar o H2 em memória por PostgreSQL no compose pra os dados não sumirem ao reiniciar, paginação nas listagens, refresh token e testes automatizados dos endpoints.

96. Por que os dados somem quando a API reinicia?
Porque o banco é H2 em memória (`jdbc:h2:mem:testdb`): ele existe só enquanto a aplicação roda. O Flyway recria as tabelas ao subir, mas vazias.